

AWS EKS Architecture Case Study

Designing a resilient ML platform for bursty GPU training, isolated data processing, and always-on inference.

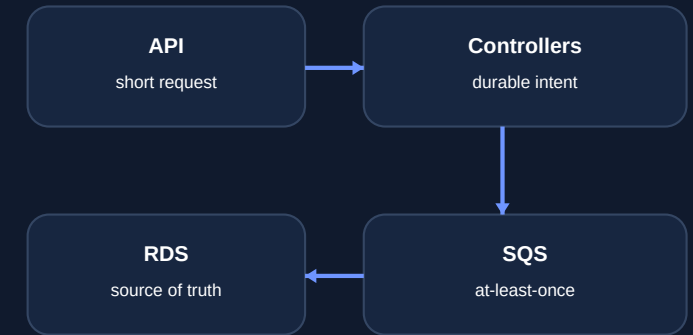
A source-grounded case study focused on workload boundaries, elastic capacity, failure containment, durable asynchronous workflows, and platform operability.

TARGET ARCHITECTURE + LIMITED API IMPLEMENTATION

DevOps / SRE / Infrastructure Engineering Portfolio

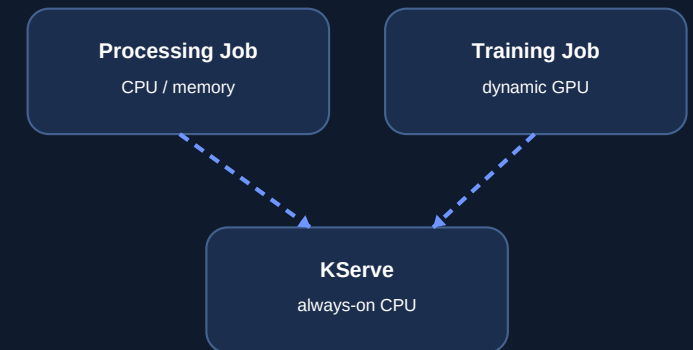
CONTROL PLANE

intent / policy / reconciliation



DATA PLANE

isolated execution / elastic compute



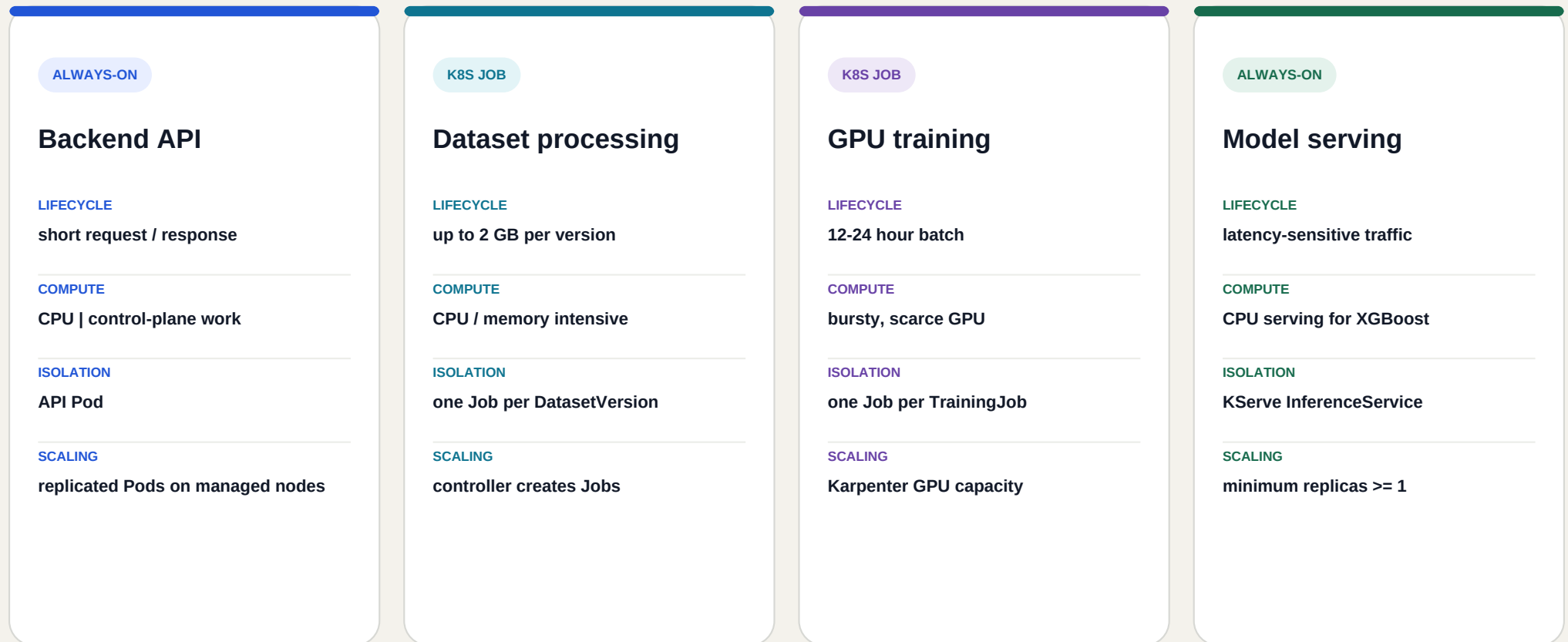
AWS EKS target architecture

Designed, not represented as production operation

01 / WORKLOAD CHARACTERISTICS

One platform, four different operating models

The architecture starts with lifecycle, resource profile, failure domain, and scaling behavior - not with a favorite service.

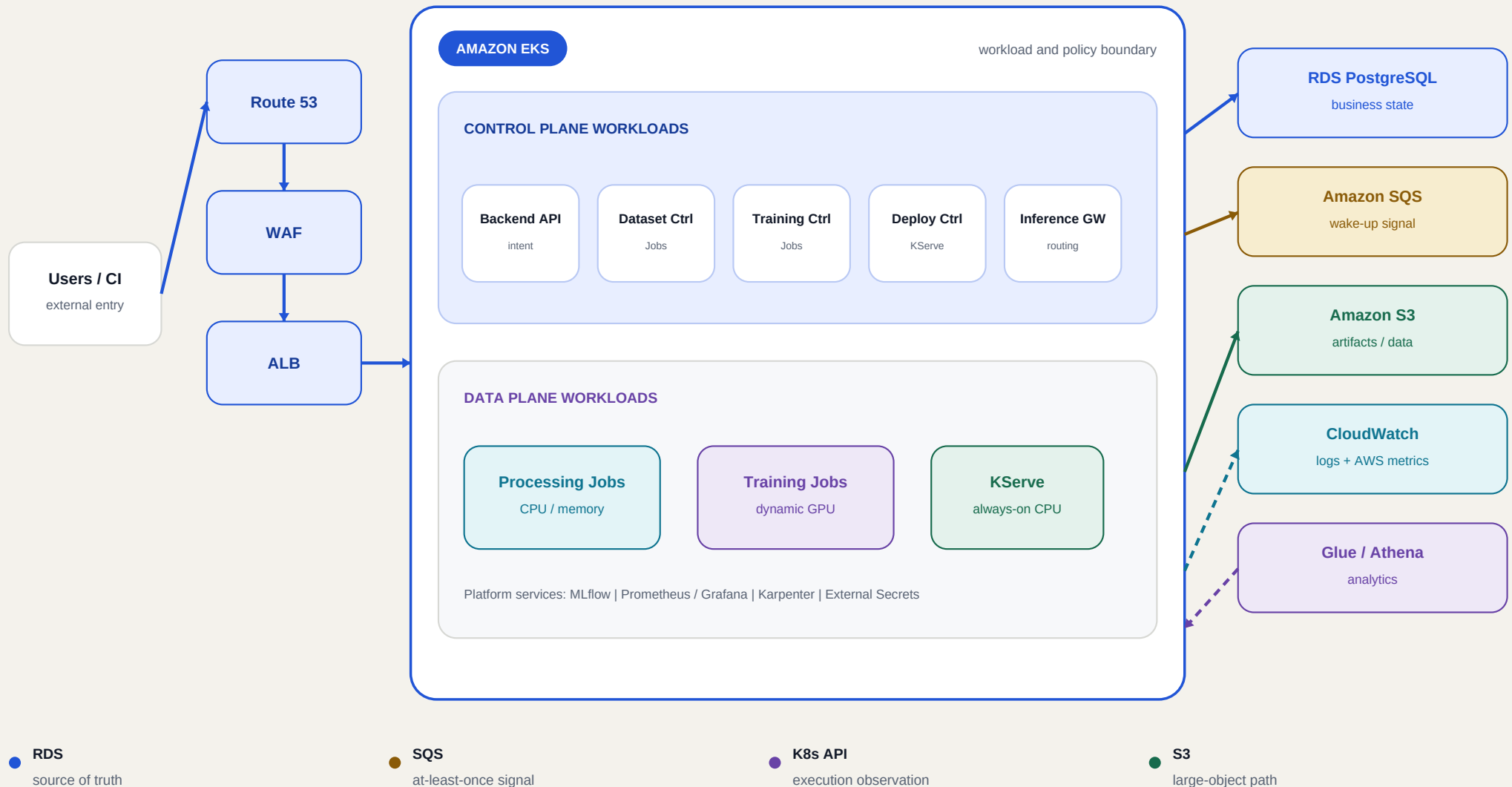


ARCHITECTURE CONSEQUENCE

Separate execution boundaries keep a 2 GB parse or a 24-hour GPU run out of the API request path.

Durable intent in PostgreSQL; isolated execution in Kubernetes

Simplified vector redraw of the repository system architecture diagram. Managed AWS services remain outside the EKS execution boundary.



Keep the platform warm; make expensive GPU capacity elastic

Karpenter is scoped to GPU training. Core services stay on a managed node group, and GPU nodes may scale to zero when no training Pod is pending.

MANAGED NODE GROUP

Always-on platform baseline

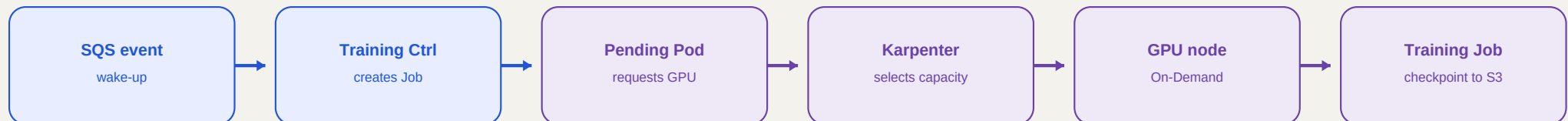
- Backend API, controllers, Inference Gateway and KServe baseline
- MLflow, observability, Karpenter Controller, CoreDNS and cluster services
- Stable capacity for latency-sensitive and control-plane workloads

KARPENTER GPU NODEPOOL

Burst capacity for training only

- Provisioned by a pending Pod requesting nvidia.com/gpu
- GPU labels, taints, tolerations and affinity prevent accidental placement
- NodePool limits cap capacity; On-Demand is the default capacity type

ELASTIC GPU PROVISIONING LOOP

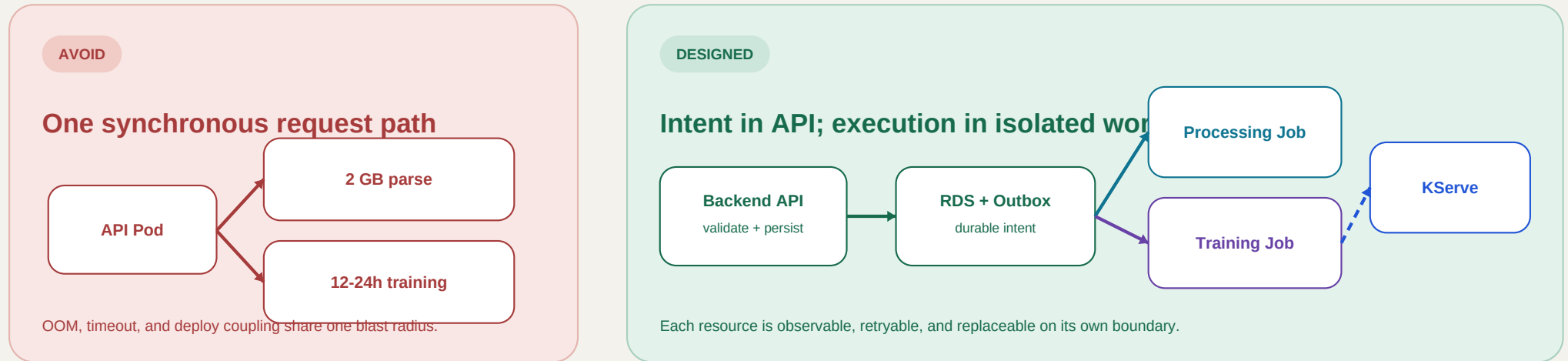


Guardrails make elasticity predictable

- No static GPU replicas
- GPU nodes can scale to zero
- Disruption / consolidation protects long jobs
- Spot only after checkpoint and retry validation
- KEDA not used: the controller already consumes SQS

Design the blast radius before designing the happy path

Large uploads and long-running training are execution workflows, not API request handlers. Each unit of work gets its own failure boundary.



FAILURE DOMAIN MATRIX

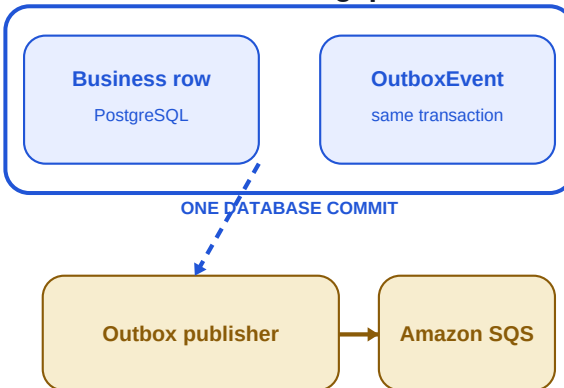
Processing runtime failure	GPU / node loss	Duplicate SQS	API rollout
BLAST RADIUS one DatasetVersion Job	BLAST RADIUS one TrainingJob	BLAST RADIUS one event delivery	BLAST RADIUS API Deployment
RECOVERY MECHANISM retry only if classified retryable	RECOVERY MECHANISM checkpoint + retry	RECOVERY MECHANISM idempotency + conditional update	RECOVERY MECHANISM training continues independently

At-least-once delivery requires state transitions that can be repeated safely

SQS wakes the platform; it is not the job store. PostgreSQL owns business intent, while the Kubernetes API reports execution state.

1 / TRANSACTIONAL OUTBOX

Remove the dual-write gap



Publisher claims unpublished rows with FOR UPDATE SKIP LOCKED.

If publish succeeds before published_at, the message may be delivered again.

2 / IDEMPOTENT TRANSITION

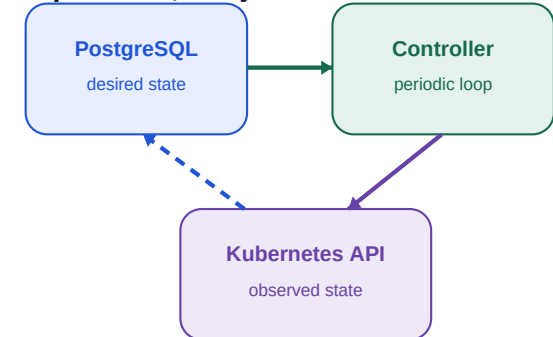
```

UPDATE training_jobs
SET status = 'RUNNING'
WHERE id = :job_id
AND status = 'SCHEDULING';
  
```

- idempotency_key on the command boundary
- event_id + resource_id in async logs
- deterministic Kubernetes resource names
- retry with backoff; never claim exactly-once

3 / RECONCILIATION

Repair drift, not just events



A periodic reconciler compares RDS intent with Kubernetes execution and repairs missed creates or stale terminal states.

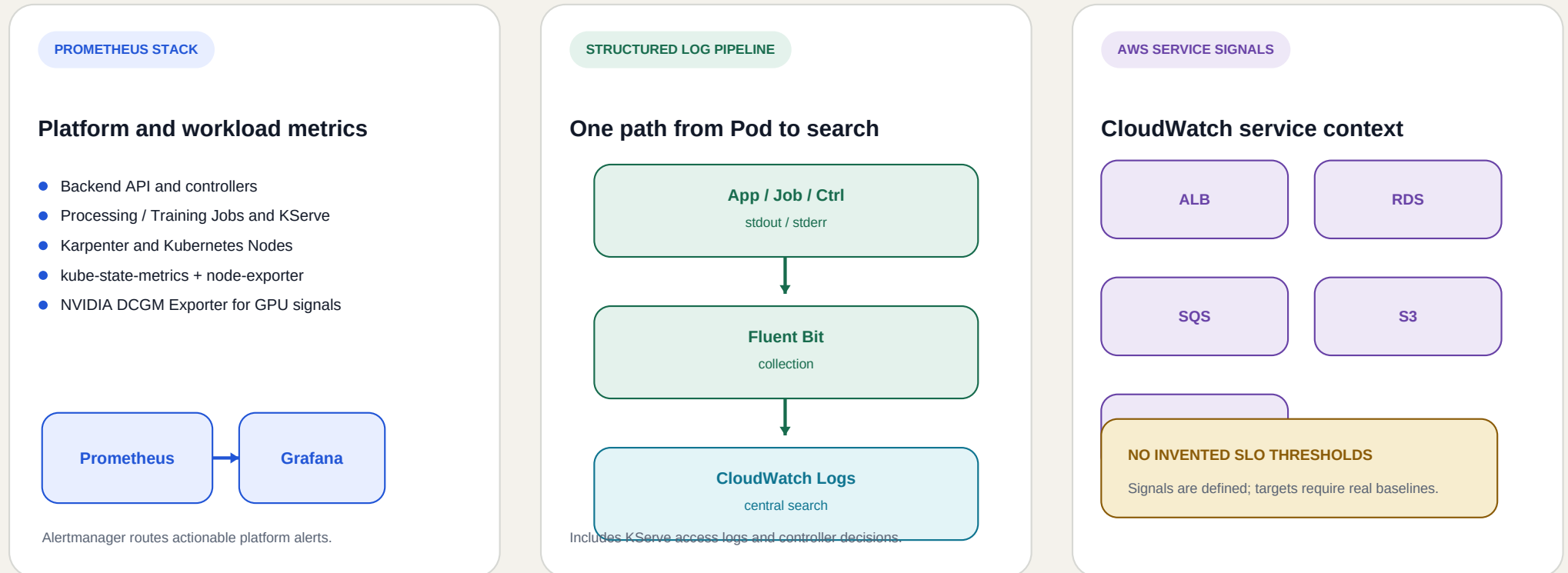
AT-LEAST-ONCE, SAFELY

Failure mode made explicit

SQS success followed by publisher crash can duplicate delivery. The design absorbs it through idempotency, conditional SQL, and reconciliation.

Correlate user intent, controller decisions, Kubernetes execution, and AWS signals

Metrics, logs, and identifiers are designed around a distributed workflow. UUIDs belong in structured logs, not high-cardinality Prometheus labels.



CORRELATION CONTRACT

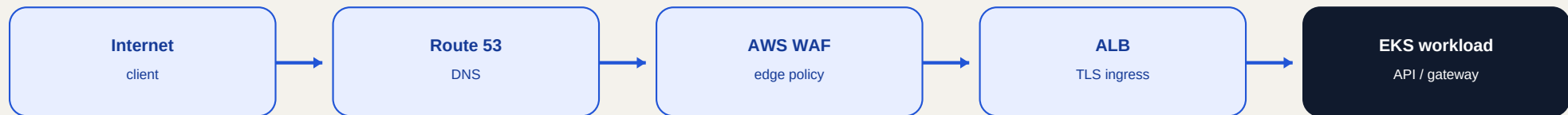
Synchronous	request_id	Asynchronous	event_id + resource_id	Structured fields	service user_id training_job_id error_code
--------------------	------------	---------------------	------------------------	--------------------------	--

Identifiers live in logs for traceability; bounded dimensions live in metrics for operational aggregation.

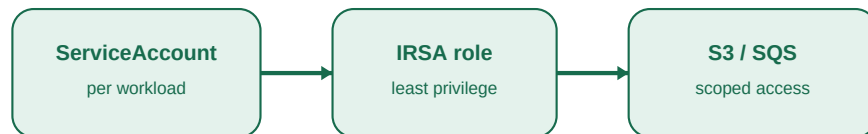
Identity and trust boundaries follow the workload, not the node

External traffic is filtered before EKS, AWS access uses workload identity, and secrets are synchronized without long-lived access keys in Pods.

EXTERNAL REQUEST PATH

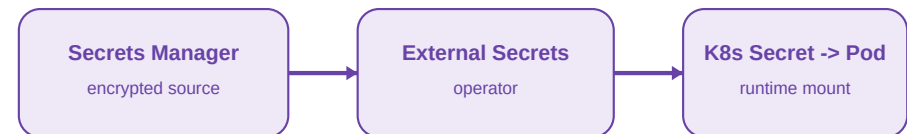


AWS WORKLOAD IDENTITY



Pods do not store long-lived AWS access keys.

SECRET DELIVERY



Only the operator reads Secrets Manager directly.

DEFENSE-IN-DEPTH CONTROLS

NETWORK

Security Groups + VPC CNI NetworkPolicy;
default-deny namespaces

POD

non-root | no privilege escalation | drop all
capabilities | RuntimeDefault seccomp

ENCRYPTION

external TLS | S3 SSE-KMS | RDS / SQS / EBS
encryption at rest

PRIVATE

RDS is not public; KServe and MLflow remain
cluster-private

A Git contract turns deployment into a reconciled platform capability

Target Architecture - Designed, Not Deployed. Developers change code and declared configuration; the delivery system converges the cluster.

DESIGNED DELIVERY FLOW



CI validates and produces an artifact. Git records desired deployment state. Argo CD owns cluster convergence.

CONTINUOUS INTEGRATION

- run tests and lint checks
- build container image and vulnerability scan
- push immutable artifact to Amazon ECR
- update declared image tag / Helm values

CONTINUOUS DELIVERY

- Argo CD watches the declared Git state
- Helm renders repeatable Kubernetes resources
- drift is detected and reconciled through one control loop
- cluster changes remain auditable through the Git history

DEVELOPER PLATFORM ANGLE

Developers interact with a reviewed Git contract - not ad hoc cluster operations.

Terraform is intentionally outside this documented delivery scope.

Add complexity only when the workload pays for it

The platform accepts operational costs where they buy isolation, durability, or elastic scarce capacity - and postpones components without a demonstrated requirement.

ADOPTED IN THE TARGET DESIGN

Amazon EKS

One scheduler and policy model for Jobs, GPU, and KServe.

COST: CLUSTER OPERATIONS

SQS + Outbox

Durable async intent without coupling work to an API request.

COST: IDEMPOTENCY + PUBLISHER

Karpenter

Elastic GPU nodes for bursty 12-24h training workloads.

COST: SCHEDULING LATENCY

PostgreSQL

Constraints and transactions provide the source of truth.

COST: RECONCILIATION DUTY

KServe Standard

Standard cluster-local serving without a Knative dependency.

EFFECT: MIN REPLICAS >= 1

NOT ADOPTED INITIALLY

Redis

No demonstrated metadata bottleneck beyond PostgreSQL.

REVISIT WITH EVIDENCE

KEDA

The controller already consumes SQS and creates Kubernetes Jobs.

AVOID DUPLICATE CONTROL LOOP

Knative

No scale-to-zero or event-streaming requirement for initial serving.

KEEP SERVING SIMPLER

Envoy Gateway

ALB + Inference Gateway + ClusterIP cover documented routing.

AVOID ANOTHER GATEWAY

CloudFront

No static distribution or download-acceleration requirement.

ADD ONLY IF WORKLOAD CHANGES

Designed architecture, implemented foundations, explicit boundary

The case study shows system-level reasoning without representing the AWS target architecture as already deployed or operated in production.

DESIGNED / TARGET ARCHITECTURE

AWS EKS platform system

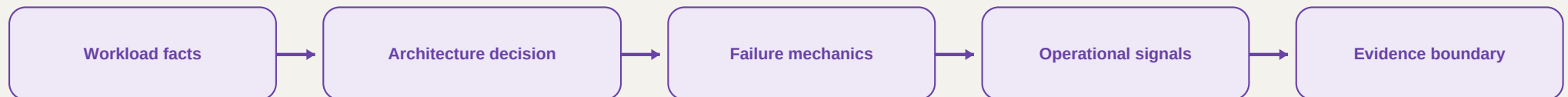
- RDS, S3, SQS and EKS service boundaries
- workload-isolated processing, training, and serving
- Karpenter dynamic GPU NodePool and guardrails
- Outbox, idempotency, conditional updates, reconciliation
- KServe, MLflow, observability, security, CI/CD and GitOps

IMPLEMENTED / REPOSITORY EVIDENCE

Limited API and data foundation

- FastAPI Dataset API: create, detail, update, delete + health
- PostgreSQL schema with 14 tables and Alembic migrations 0001-0004
- transactional audit logging, soft delete, aggregate query behavior
- Docker Compose development path and automated tests
- No claim of deployed EKS, Karpenter, KServe, MLflow, or GitOps

DECISION LINEAGE



LIVE ARCHITECTURE PORTFOLIO

<https://naladoodong.github.io/Kubernetes-EKS-MLOps/>

SOURCE REPOSITORY

<https://github.com/naladoodong/Kubernetes-EKS-MLOps>